Composing certificate refinements and arithmetic summaries

Further simplification research

21 October 2026

Abstract

Recent commits have actually adopted an ordinary source checker, while newly delivered archives separately certify normalized and balanced variants. The nearest incremental Rocq proposal composes balanced lookup with normalized checking: all 1,426 entries and 2,851 branches pass independent checks, with 32,961 lookup visits instead of 2,033,059 list visits. This is an operation count, not a measured build speedup; the connecting Rocq proof is uncompiled. A strategic arithmetic interface contracts 174 triangular loops and four conditional-clear loops to 189 control nodes. Ten bounded edge regressions remain explicitly uncompleted. Further proposals reuse an existing quotient satisfaction theorem and isolate a missing append-zero runtime contract. No new machine, kernel build or state reduction is claimed.

1 Recent commits change the baseline

The reviewed main pin is

`aa66d0a33f1f8237a348899eb8095b045a502832`

Unlike the preceding archive-placement rounds, phase 5 adopts ordinary source certificates and shared semantic lemmas in maintained code. Its source checker retains 2,240 entries but drops their zero offsets. It explicitly uses 512-fuel decoding, not the legacy 128-fuel specialization that changes 622 entries. Sixteen certificate chunks were checked, and source data decreased from 456,548 to 432,131 bytes. Maintained code does not yet use 1,426-entry normalization or balanced lookup.

The phase-5 record covers a 184-module endpoint dependency closure incrementally: 37 fresh compilations and 147 inherited objects bound to source, dependency, object and log evidence. Two typed endpoint clients and ten standard axiom reports pass, with zero audit warnings and 31 selected source-linter warnings. Every selected compiler/audit exit is zero. The outer execution nevertheless reported exit 143 after its saved PASS/final marker; its cause is not established. This must not be described as a successful outer-shell exit or a wholly fresh rebuild. These are inspected upstream records, not local compilations.

Shared semantic changes shorten an equivalence induction and remove repeated impossible-tag cases. The model file targeted by the previous three-induction patch is unchanged, so that proposal remains unadopted. The four concrete source-program/tree definitions are textually unchanged, binding the earlier graph analysis to the current source without implying unchanged proof dependencies.

New archives report: a Coq 8.18 normalized 1,426-entry source theorem with recursive `coqchk`; 281 literal decoder-block proofs whose source-path and primitive joins remain open; concrete zero-credit codec/injection proofs without a physical adapter; and a compiled balanced 2,240-entry checker. These are distinct scopes, not complete alternative machine endpoints. Independent raw hash inspection finds

24/24 selected compiled sources in the checked archive and 29/30 in the refinements archive. The unmatched refinement dependency is a documented Coq 8.18 compatibility-adapted `RMEquiv.v`; its delivered upstream source differs. Do not call 29/30 complete exact compiled-source verification or an unexplained theorem failure.

An additional automatic-certificate archive records the same inferred 1,426 tuples accepted by both normalized checkers. Its Lean evidence covers 30 fresh project modules and three emitter executions, with same-build resource retries and no preexisting project objects. All 33 source and log records match included bytes. Its Rocq evidence covers two new modules against ten hash-bound dependencies, without a second recursive audit; both new source/log pairs match. Shared entry files are byte-identical. These records do not rebuild a full machine or ZFC endpoint. Automatic generation is therefore already demonstrated, not an open novelty claim.

2 Compose Rocq balanced lookup with normalized equality

Automatic-certificate evidence inspected during this review already combines normalization with inherited balanced certificate blocks on the Lean side. The idea of balancing normalized data is therefore not new in general. The concrete contribution here is an independently checked *Rocq* composition of the new balanced lookup interface with normalized branch equality; the inspected automatic Rocq lane still uses list-based `NCheck`.

The two archived Rocq refinements address different costs. Normalization removes implicit trailing-zero pointer aliases, reducing correspondence to 1,426 entries. Balanced storage reduces indexed lookup cost. Their existing theorems cannot simply be interchanged: the balanced checker refines literal-path equality, whereas normalized branches require equality of infinite zero-padded paths, `ibeq`.

The proposed small Rocq glue keeps the existing valid-tree lookup and flattening theorem. It changes only branch/entry wrappers so that a branch checks increment counts and two normalized endpoint equalities. Its key refinement is

$$\mathrm{nbcheck}(t_1, t_2, T) = \mathrm{ncheck}(t_1, t_2, \mathrm{flatten}\, T) \quad \text{when } T \text{ is valid.}$$

The source-halting theorem then follows from the existing normalized checker's soundness. No new path semantics, segment evaluator, or simulation induction is needed. The glue, exact identifiers and concrete instance remain **uncompiled**.

The owned complete finite check parses the current source-tree literals and the archived packed path encoding, checks every index lookup against list lookup, and checks all 2,851 branches including initialization. It obtains:

Normalized entries	1,426
Balanced node visits	32,961
Linear-list visits for the same lookups	2,033,059
Maximum visits per balanced lookup	12
Literal endpoint comparisons that fail but satisfy `ibeq`	2,659

Four malformed tree/index/entry mutations are rejected. The 61.68-fold ratio is only a reduction in this defined lookup-visit count, not kernel time, memory, proof size or whole-build time. Tree construction, validity and segment evaluation remain work; existing evidence already shows nontrivial construction cost. Emit a literal tree and benchmark the entire pipeline.

The concrete next step is to compile the glue against the pinned modules, generate the literal tree, kernel-check validity and flattening, run the normalized balanced check on the actual programs, and

retain the public all-register source theorem. Then update the generator and rebuild affected endpoint clients with assumption reports. Hash equality is provenance, not a replacement for any of those kernel checks.

3 A guarded 189 node arithmetic interface

The previous 367-node affine core contains 174 copies of one triangular loop and four conditional-clear loops. Exact recognition and ranked contraction leave 189 nodes: 188 tests and one halt, with 376 edges. Updates are now short ordered programs of affine matrices and guarded summaries, not unrestricted polynomial expressions. The resulting JSON is slightly larger than the preceding affine representation despite fewer control nodes. This is a reusable semantic interface, not byte compression, a smaller Turing machine or a demonstrated faster proof.

3.1 Triangular recurrence with arbitrary scratch

Every recognized triangular node tests register R_6. Put $x = R_6$, $s = R_1$, $y = R_7$. Its nonzero branch first decrements x, then its exact matrix yields

$$(x, s, y) \longmapsto (x + s - 1,\ 0,\ y + x + s) \qquad (x > 0).$$

All other registers are unchanged. If $x = 0$, the body is skipped: scratch s is preserved and nothing is added to y. If $x > 0$, let $n = x + s > 0$. The first iteration clears scratch and adds n, leaving $x = n - 1$; the remaining iterations add $n - 1, n - 2, \ldots, 1$. Thus the pre-exit summary is

$$\begin{cases} R \text{ unchanged}, & x = 0, \\ R_6 = 0,\ R_1 = 0,\ R_7 = y + T(x + s), & x > 0, \end{cases} \qquad T(n) = n(n + 1)/2.$$

The zero-exit matrix is applied *after* this summary and may modify any register. The contribution is $T(x + s)$, not $T(x + s) - T(s)$: $x = 1, s = 5$ adds 21, not 6. At $x = 0, s = 5$, it adds zero and preserves five. A scratch-zero invariant is neither needed nor assumed.

The universal argument separates the first nonzero iteration from the scratch-zero countdown and uses $T(k + 1) = T(k) + k + 1$. It consumes exactly $x + s$ nonzero steps and one zero exit when $x > 0$, or just the zero exit when $x = 0$. A claim that every iteration decreases R_6 would be false: the first iteration may increase it when scratch is nonzero.

3.2 Conditional clear and composition

The four other loops decrement R_6 and clear R_5 on each nonzero iteration. Their summary preserves all registers if $R_6 = 0$; otherwise it sets $R_6 = R_5 = 0$ and preserves the rest. Unconditionally clearing R_5 would be wrong on a zero entry. They consume the old R_6 nonzero steps and one exit.

Every removed node has a checked rank that strictly decreases when its exit is another removed node; maximum rank is seven. Each retained nonzero branch explicitly decrements its tested register *before* its edge program. Matrices, summaries and zero-exit updates follow their literal order. All local summaries terminate on arbitrary natural registers and consume at least one step; each retained edge also has its own test step. No halt is eliminated. These facts support both halting implications and preservation of final registers, with the original retained initial node and identity entry transform.

For the present pair of identical canonical source graphs, direct normalized or canonical equality is still simpler than this additional contraction pipeline. Use the arithmetic interface when comparing

genuinely different loop implementations or sharing a source-level arithmetic proof, not merely because the displayed node count is lower.

3.3 Completed checks and ten explicit limits

The finite validator checks every exact matrix shape, branch polarity, exit, rank, retained-node partition and edge instruction list. All such checks complete. It also reports 14,418 successful local executions and 554 successful complete edge executions, with four adversarial certificates rejected.

Ten further edge-regression inputs exhausted their 10,000-step budget. They are not passed, failed, or proved divergent. Composed triangular summaries can grow register values rapidly. The all-natural argument above is separate from bounded simulation and does not depend on pretending these ten runs finished. The exact audit preserves them as uncompleted. Generic summary semantics, source-to-graph adequacy and contraction soundness remain proof-assistant obligations.

4 Reuse the quotient theorem and the generator

The existing `relSat_congr` duplicates a formula induction even though the same module already proves `sat_quotient`. Move congruence after that theorem. Pointwise related environments have equal quotient images by `mk_eq_iff`; function extensionality gives equality of quotient environments. Rewriting both sides through `sat_quotient` proves the original congruence statement. Its dependency graph is acyclic: the quotient theorem does not use the congruence being removed.

The proposed parent file drops from 216 to 192 lines. The generated `KQ1Model` has the same opportunity and also duplicates 54 lines of model-independent syntax and two structural inductions. Replace that block by 17 lines of aliases/wrappers to the existing parent syntax. The generated file drops from 219 to 159 lines; combined Lean-source reduction is 84 lines, not 84 net repository lines, because the generator grows. No object-size or speedup claim follows.

Apply the parent patch, then change the generator and regenerate. The aliases must be inserted *after* its namespace transformation; inserting them earlier would rewrite parent references back to self-referential aliases. The owned tool only rewrites inert text. Neither the received generator nor the proposed modified generator was executed, and the Lean files remain uncompiled. Exact regeneration, idempotence and dependent endpoint compilation are next checks.

A further quotient-core interface needs only reflexivity, symmetry and transitivity of the existing relation; extensionality is added only where used. Larger model hypotheses and a witness remain necessary later for actual model existence and the final consistency theorem. Sharing helpers must not silently remove those endpoint assumptions or change the quotient's domain presentation. A new shared module may also add build dependencies, so fewer source lines need not mean a faster build.

5 Close the append-zero runtime boundary

The maintained `KTape.boxLemmas` already factors its engine through agreement on 44 rows plus an increment-tail contract. ST215 and DR215 agree on all 44 rows, and their two changed decoder rows lie outside that set. This is existing infrastructure, not a new scanning theorem.

There is nevertheless an exact missing premise: maintained `IncTail` and the concrete box interpretation append spare value one, while ST/DR use the row 198/0 = 0L40 and append zero. Row agreement

alone does not instantiate the maintained engine theorem. The earlier dependency checks correctly left box lemmas assumed; they cannot be upgraded to a physical proof by ignoring this difference.

Parameterize the box interpretation and increment postcondition by a fixed Boolean spare value. Generalize the final tail segment once, then thread that parameter through concrete interpretation, oracle/step/executor soundness and the simulation extension witness. Other box contracts are unchanged; arbitrary suffix return lemmas already exist. The ordinary K215 instance uses one and ST/DR use zero. Replacing only the final witness is insufficient because the whole tape interpretation currently fixes the spare value.

An independent six-step local tail check starts at state zero, head at the first of four zero cells, and covers both spare values. Every read stays within that window; it ends at state 40, head back at its first cell, with exact contents $[1, 0, 1, a]$, preserving arbitrary exterior tape. This isolates a small proof obligation but is not the parameterized kernel theorem. The owned access-logged audit confirms exact symbolic-state and progress-flag equality for 6,012 unchanged guards, freshly checks 139 affected guards, and exercises all 36 oracle interfaces. A generic trace-dependency proof must compare every actual table lookup, including absent-row results and failed-oracle fallbacks, and combine with the correctly parameterized box soundness. Until then, these are conditional reuse opportunities, not a complete DR physical theorem.

6 Reproduction and priorities

The packet includes owned offline checkers, full pinned input/certificate data, prior graph/drain replay dependencies, current review receipts, uncompiled glue and generator patches, and inherited source notices. Run with Python 3 and assertions enabled, using a fresh external directory:

```
python -I -B verify.py --work /tmp/round10-check
```

The replay regenerates graph/drain/arithmetic data, balanced normalized checks and mutations, proposed generated text, and runtime dependency/engine checks. Its success explicitly retains the ten exhausted arithmetic regressions. It does not compile Lean/Rocq or rerun archived compilers.

Prioritize the balanced-normalized glue and exact source instance, the quotient/generator refactor, and the spare-parameter runtime theorem. Keep arithmetic summaries as a strategic interface for differing implementations. Preserve public theorem statements, assumption reports and exact table attachments; measure complete builds separately. No new calibration strength, internal-PA theorem, state reduction or measured build-speed claim is made.